

Unified Context Engine: An On-Premise, Policy- and Requirement-Aware LLM Assistant

CS 540 Final Project Report

Preet Patel
May 6, 2026

This report follows the CS 540 final project outline and summarizes the motivation, research, approach, implementation, empirical results, deliverables, and future work for the Unified Context Engine (UCE).

AI Tools Usage:

I acknowledge the use of OpenAI codex-5.3-high to draft the report, and Anthropic Claude Sonnet 4.6 to create the architecture diagrams. This project exhaustively utilized the OpenAI codex-5.3 model under the supervision of the author.

Acknowledgement:

I would like to express my sincere gratitude to Professor Ugo Buy for his invaluable support during the conceptualization of this project and for providing the API credits necessary to facilitate this work. Additionally, I am deeply thankful to Professor Mark Grechanik for his mentorship and for the foundational knowledge of knowledge graphs gained in his CS441 course, which proved essential to the implementation of this project.

Disclaimer:

This project is a localized proof of concept designed to establish a foundational framework for future development. Due to hardware constraints, the implementation relies on local deployment of efficient open-source models, specifically Llama 3 Instruct, rather than cloud-hosted alternatives. Consequently, exhaustive benchmarking against commercial LLMs was outside the current scope, with evaluation limited to preliminary comparisons.

A core architectural assumption of this work is that enterprise environments handling proprietary data face significant privacy risks when utilizing external cloud APIs. Therefore, this prototype intentionally prioritizes a strict on-premise, local infrastructure model to guarantee data sovereignty.

1. Introduction

The original proposal for this project was titled "An On-Premise, Organization-Bound LLM Assistant for Context-Aware Software Engineering." The central idea was to build a local LLM assistant that could answer software engineering questions using private organizational context rather than relying only on public training data. The project began from a practical gap in current developer assistants: tools such as ChatGPT, Claude, and Copilot can reason over general programming knowledge, but they usually do not know a company's private codebase, policies, data models, architecture decisions, internal documentation, or access-control rules unless that information is manually pasted into a prompt or sent to an external provider.

During implementation, the project evolved from a general context-aware assistant into a more focused system: the Unified Context Engine, or UCE. UCE builds a deterministic graph over code, schema, requirements, policies, and RBAC authority rules, then exposes graph-backed reasoning and gated mutation tools through an MCP server. This modified direction preserved the original privacy and organization-bound motivation, but made the technical problem sharper: the goal became not merely to retrieve useful context, but to make LLM-assisted software engineering accountable to requirements, policies, and role-based authority constraints.

A. Original goals of the project

The original goal was to create an on-premise, organization-bound LLM assistant for software engineering. The assistant was intended to run locally, ingest a representative internal software project, and answer developer questions that a generic cloud LLM could not answer because the required information lived in private source code, internal documentation, logs, or local tools. The proposal emphasized three layers: a local LLM core, a knowledge retrieval layer using RAG and GraphRAG-style structured retrieval, and an MCP-style integration layer for accessing tools and repositories through standardized interfaces.

The original expected outcome was a proof-of-concept assistant that could answer internal development questions with citations to code or documentation. Example target questions included locating API ownership, explaining build failures in the context of a local CI/CD setup, and finding internal usage rules for custom libraries. In short, the original project asked: can a local assistant become useful by grounding answers in organization-specific context while keeping private data on-premise?

B. Modified goals, why modified

The goals changed because context-aware assistance alone does not solve the more important software-engineering risk: LLMs can produce plausible but policy-violating, requirement-incomplete, or authorization-bypassing suggestions. In practice, a developer assistant needs to know not only where code lives, but also what requirements the code satisfies, what policies govern those requirements, and who is allowed to make particular changes. A longer prompt can help in simple cases, but prompt curation remains non-deterministic. It can miss constraints, overfit to keywords, or produce inconsistent answers across runs.

The modified goal was therefore to build a local, policy- and requirement-aware context engine. Instead of relying on keyword matching or prompt-only reasoning, UCE represents governance facts as graph nodes and edges. Requirements govern tables and columns; policies enforce requirements; files reference schema entities; imports create transitive impact; and RBAC rules define which roles may read, write, or delete protected paths. This made the project more research-oriented: the final system compares a no-tool local LLM baseline against MCP-UCE, measuring whether a model catches policy/requirement violations and whether it breaches RBAC constraints.

The privacy motivation also became stronger. Current cloud LLMs are powerful, but sending proprietary code, schemas, policies, and access rules to external systems raises data privacy and intellectual-property concerns. Because this project targets organization-bound software engineering, I chose to evaluate with a local, small LLM, `llama3:instruct`, rather than framing the system around large hosted models. This choice matches the original on-premise goal and also reflects hardware constraints typical of student and small-team environments.

C. Assumptions you made

The implementation and experiments were conducted on a Windows development machine using PowerShell. The UCE implementation is primarily Python, with the target project written largely in TypeScript/Next.js using a

Drizzle database schema. The graph backend is Neo4j, authentication and role-token minting are represented through Keycloak, and the tool interface follows the Model Context Protocol pattern. The local LLM baseline used `llama3:instruct` through a local OpenAI-compatible endpoint. The evaluated target repository was a representative software project with source files, database schema files, requirement Markdown documents, policy Markdown documents, and RBAC Markdown rules.

- Programming languages used: Python for UCE, TypeScript/JavaScript for the target application, Markdown for governance artifacts.
- Operating environment: Windows with PowerShell, local Python 3.12 tooling, local graph/service processes, and local LLM inference.
- Primary datastore assumption: Neo4j can store a deterministic knowledge graph over code, schema, requirements, policies, functions, imports, identifiers, and RBAC rules.
- Security assumption: the assistant should connect to UCE MCP rather than directly to the raw graph backend, because direct graph access can bypass policy and RBAC checks.
- Evaluation assumption: exact requirement IDs, policy IDs, file paths, and RBAC allow/deny decisions are appropriate measurable outputs for comparing no-tool LLM behavior against MCP-UCE behavior.

D. Definitions of terms specific to your project

UCE: Unified Context Engine, the system built in this project. It creates and serves a governance-aware graph over code and software artifacts.

MCP: Model Context Protocol, a standardized pattern for exposing data sources and tools to AI assistants through structured interfaces.

GraphRAG: A retrieval approach that uses graph structure, not only flat text chunks, to retrieve connected context for LLM reasoning.

Requirement: A project-level rule describing what the software must preserve, such as required schema fields or audit properties.

Policy: A governance artifact that enforces one or more requirements.

RBAC: Role-Based Access Control. In UCE, RBAC rules define whether viewer, editor, or admin roles may perform read, write, or delete operations on paths.

No-tool LLM baseline: A local LLM that receives pasted static context only and cannot query UCE, Neo4j, files, grep, or MCP tools.

MCP-UCE: The graph-backed, MCP-exposed UCE system that performs deterministic impact analysis and RBAC-gated mutations.

E. Summary of approach that you followed

The final approach was to turn software context and governance context into a single graph. Code ingestion parses files, imports, functions, classes, method declarations, and identifiers. Schema ingestion parses tables and columns. Requirement ingestion links requirement IDs to tables and columns through deterministic evidence. Policy ingestion links policies to the requirements they enforce. RBAC ingestion converts authority rules into enforceable role/path/operation constraints. The resulting graph supports queries such as: if a table or column changes, which files, requirements, and policies are implicated? If a user with a given role tries to mutate a path, should the system allow the action?

This strategy separates two jobs that are often blurred together in LLM applications. The LLM can still explain, summarize, and interact conversationally, but UCE handles the source-of-truth reasoning for impact, requirements, policies, and authority. This is the main design contribution: use the LLM as an interface and reasoning partner, but use deterministic graph relationships and RBAC gates for decisions that must be auditable.

F. Project deliverables

- A Python-based UCE ingestion pipeline for code, schema, requirement, policy, and RBAC artifacts.

- A Neo4j graph schema covering files, tables, columns, requirements, policies, functions, classes, methods, identifiers, roles, and authority rules.
- MCP-exposed reasoning tools such as impact analysis, explanation, risk assessment, identifier usage, and function counting.
- RBAC-gated mutation tools including `authorize_change`, `write_file`, and `delete_file`.
- Keycloak-based role-token bootstrap support for viewer, editor, and admin roles.
- A benchmark comparing a real no-tool local LLM baseline using llama3:instruct against MCP-UCE on requirement/policy capture and RBAC breach behavior.
- Architecture diagrams, evaluation graphs, tables, and this final report.

G. Contribution of individual team members

This was an individual project. I, Preet Patel, was responsible for the proposal, literature review, system design, implementation, debugging, evaluation, benchmark design, results analysis, documentation, diagrams, and final report.

2. Background research that you did

The background research combined primary research from an industry practitioner with secondary research on LLMs, retrieval-augmented generation, graph-based retrieval, tool use, MCP, and AI governance. The most important takeaway was that most deployed LLM workflows rely heavily on prompt curation and human review, while this project needed a more auditable mechanism for software-governance constraints.

A. Primary research

For primary research, I discussed the problem with a senior program manager working at Discord. I asked how teams make sure LLM-driven or AI-assisted engineering workflows do not violate policies and requirements. The answer was practical and realistic: teams curate prompts, enhance prompts with more context, rely on manual verification and human-in-the-loop review, and sometimes use an LLM-as-judge to evaluate outputs. However, the LLM-as-judge pattern is not always accurate, and manual review does not scale cleanly when the number of requirements, policies, and code paths grows.

This conversation shaped the project direction. If the standard answer is better prompts plus manual checking, then there is room for a system that represents requirements and policies as first-class artifacts. Instead of asking an LLM to remember all constraints from a prompt, UCE gives the assistant a governed context engine that can compute affected files, requirements, policies, and authorization decisions deterministically.

B. Secondary research

The secondary research began with general LLM capability papers. Brown et al. showed that large language models can perform many tasks through few-shot prompting, but this also highlights a limitation: prompt-based behavior is flexible but not inherently grounded in private organizational state [1]. Chen et al.'s Codex work showed the promise of models trained on code, motivating the software-engineering assistant direction, but such models still rely on public training data and do not automatically know a private codebase [2].

RAG research was directly relevant because the original proposal focused on local context retrieval. Lewis et al. introduced retrieval-augmented generation as a way to combine parametric language models with non-parametric retrieved documents [3]. GraphRAG extended that idea by using graph structure to represent entities and relationships rather than retrieving isolated text chunks only [4]. For this project, that insight became central: software systems are not flat documents. They are networks of files, imports, functions, schemas, requirements, and policies.

MCP research and documentation were also important. Anthropic describes MCP as an open protocol for connecting AI assistants to the systems where data lives, replacing fragmented custom integrations with a consistent tool/data interface [5]. This matched the project goal of exposing UCE as a controlled gateway rather than giving an assistant direct database or filesystem access. Related tool-use papers such as ReAct and Toolformer also support the broader idea that models become more useful when they can call external tools, but the tool layer must be constrained and auditable [6], [7].

Finally, AI governance work such as NIST AI RMF reinforced the need to treat AI system behavior as a risk-management problem, not merely a model-quality problem [8]. For UCE, this translated into measurable governance outcomes: did the assistant catch implicated requirements, did it identify enforced policies, and did it avoid RBAC breaches?

C. What was most helpful to your goals?

The most helpful input was the combination of MCP and the Discord primary research discussion. MCP suggested the architectural boundary: the assistant should interact through a controlled tool server. The primary research showed why prompts and human review are not sufficient as the only guardrail. GraphRAG and RAG literature helped explain why structured retrieval is better than flat keyword matching for connected software systems. Together, these sources motivated the final strategy: keep the LLM local and small for privacy and hardware reasons, but compensate for model limitations with deterministic graph context, explicit requirements, policy edges, and RBAC gates.

3. Approach to your goals

A. The strategy you followed

The strategy I followed was to separate two responsibilities that are often mixed together in AI developer tooling: language generation and governance reasoning. In many assistant workflows, an LLM is asked to both understand free-form engineering requests and to enforce requirements, policies, and authorization constraints. That combination is risky. Language models are excellent at synthesis, but they are not deterministic policy engines. So the first strategic decision in UCE was architectural: the model can explain and interact, but a deterministic context engine must own impact analysis and governance-critical decisions.

This decision directly shaped how context is represented. Instead of relying on long prompts that include partial code snippets and prose rules, UCE builds a graph where software entities and governance entities are first-class nodes. Files, functions, tables, columns, requirements, policies, roles, and authority rules are connected through typed edges. This means the system can answer governance questions through explicit traversals rather than through model guesswork. If a user asks about changing a column, the system does not rely on the model remembering which requirement might mention it; it follows graph links with deterministic queries.

A second strategic choice was to emphasize deterministic ingestion first, then optionally layer LLM-assisted extraction. Deterministic parsing gives strong reproducibility for code and schema, while governance documents can still be interpreted in a structured way. Optional LLM extraction exists for underspecified documents, but the baseline architecture does not depend on LLM extraction quality to preserve core safety properties. This was important for two reasons: first, reproducibility in an academic setting; second, operational reliability in environments where model behavior may vary across runs.

A third choice was to treat governance artifacts as implementation inputs rather than documentation side-notes. In many projects, requirements and policies exist as markdown files that are read by humans but not connected to the runtime decision process. UCE explicitly ingests those artifacts and links them to technical entities. The effect is practical: an impact result can include implicated requirement IDs and enforced policy IDs, creating a chain of evidence from proposed code change to governance risk.

A fourth strategic choice was to enforce a strict MCP boundary. The assistant should call UCE MCP tools, not raw graph endpoints. This boundary is not merely an integration preference; it is a security control. If an assistant can directly mutate a backend system, it can bypass policy checks even if the prompt says it should not. By funneling mutations through guarded tools, UCE ensures authorization checks happen at execution time, not only at suggestion time.

A fifth choice was to evaluate the system with safety metrics, not only relevance metrics. Typical AI evaluations emphasize helpfulness, BLEU-like text overlap, or subjective quality. Those are insufficient for governance-heavy software engineering. UCE therefore evaluates requirement capture, policy capture, and RBAC breach behavior in addition to overlap-based impact quality. This aligns the evaluation with the stated objective: safer organization-bound engineering assistance.

In implementation terms, the strategy can be viewed as a layered control plane over software change reasoning. The ingestion layer builds structured state. The reasoning layer answers impact and risk queries from that state. The authorization layer evaluates whether a change may proceed for a specific role and path. The tool exposure layer presents these capabilities through a stable MCP interface. Each layer has a narrow contract, which reduced coupling and made the system easier to debug and evaluate.

I also intentionally selected scenario types that reflect real developer work: table changes, column changes, and file-level refactors. These are common operations that require cross-domain reasoning because schema and code dependencies are rarely isolated. This scenario design forced the system to prove it could combine multiple contexts instead of succeeding on single-document retrieval.

Finally, I treated auditability as a functional requirement, not a bonus. Every major design element in UCE was chosen to preserve traceability: deterministic links, explicit IDs, typed relationships, and explainable rule decisions. If an output says a path is denied, the decision should be attributable to a rule and role relation, not to

an opaque model confidence score. This was central to the strategy because enterprise adoption depends on post-hoc explainability to technical and non-technical stakeholders.

Strategy Summary (Operational Steps)

1. Ingest code, schema, requirements, policies, and RBAC documents into a deterministic graph.
2. Expose graph-backed reasoning tools through UCE MCP.
3. Prevent direct Neo4j-MCP access from acting as an authorization bypass.
4. Evaluate a no-tool local LLM baseline against MCP-UCE using exact IDs and RBAC probes.
5. Use results to measure not only relevance, but policy/requirement capture and RBAC safety.
6. Preserve evidence paths so each high-impact or denied decision can be explained and audited.
7. Keep local-first execution and data boundaries to align with private engineering environments.

B. Motivation for your strategy

The primary motivation was that policy and requirement compliance needs guarantees stronger than prompt compliance. Prompt instructions are soft constraints; they can be ignored, misinterpreted, or outweighed by other tokens in the model context window. In contrast, software governance requires hard constraints. If a path is protected by policy, a denied role should be blocked every time, regardless of prompt wording. This difference between soft linguistic guidance and hard execution controls motivated the entire UCE architecture.

A related motivation was operational trust. Teams need to justify why a proposed change is considered risky, why a requirement is implicated, or why an operation is denied. Without structured evidence, AI outputs become difficult to defend in code review, compliance review, or incident retrospectives. The graph approach addresses this by producing inspectable links among code entities and governance entities. A report that includes `RQ-001` and `P-001` is no longer a vague warning; it is a traceable claim anchored to known artifacts.

Privacy was also a first-order motivation from the beginning of the project. The original proposal emphasized on-premise or organization-bound operation so proprietary code, schema, and internal policy documents are not routinely sent to external services. That motivation remained intact in the final implementation. The benchmark includes a local small model (`llama3:instruct`) not because it is universally best at language tasks, but because it reflects the practical deployment constraints of privacy-sensitive and resource-constrained teams.

Another motivation was reproducibility. If two identical inputs can produce materially different governance decisions, the system becomes difficult to validate and difficult to improve. Deterministic components provide a stable baseline for debugging and comparison. By fixing graph relationships and rule evaluation logic, experimental differences become easier to attribute. This was particularly important in the course context, where the project had to show not only implementation novelty but also credible empirical methodology.

The strategy was also motivated by the limitations of flat retrieval in connected software systems. Source code dependencies, database schema constraints, and policy mappings form a relational network, not independent text chunks. A change in one location can trigger transitive effects through imports, function calls, schema references, and requirement links. Graph representation is better suited to this structure because it makes multi-hop dependencies explicit and computable.

Security and least-privilege concerns provided additional motivation. In practical assistant deployments, the dangerous failure mode is not merely an incorrect explanation; it is an unauthorized action that actually executes. UCE therefore treats mutation authorization as a runtime gate. Even if an LLM suggests an edit confidently, the operation still passes through role- and path-aware rule checks. This shifts safety from advisory warnings to enforceable controls.

A final motivation was to produce results that are meaningful to engineering leadership and governance stakeholders, not only to ML practitioners. Metrics such as requirement caught-any rate, policy caught-any rate, and RBAC breach rate directly communicate whether an assistant supports compliant engineering behavior. These metrics are easier to interpret in organizational settings than generic language quality measures. The approach therefore aligns technical architecture, evaluation design, and stakeholder communication around the same governance objective.

In summary, the motivation can be expressed as a simple principle: use LLMs for language and interaction, but use deterministic systems for policy-critical decisions. UCE operationalizes that principle with graph-based context, MCP-mediated access, and enforced RBAC at mutation boundaries. The resulting system is intentionally conservative where safety matters and flexible where conversational productivity matters.

C. How you divided the work among team members

Because this was an individual project, the work was divided by project phase rather than by team member. I first wrote the proposal and researched RAG, GraphRAG, MCP, and local LLM deployment. I then implemented ingestion and graph schema support, followed by MCP reasoning tools and RBAC-gated mutation tools. After that, I built the benchmark, corrected the baseline to use actual no-tool LLM responses, generated evaluation figures and tables, and prepared the final report.

4. Implementation

A. Design of your software, if any

UCE is designed as a guarded middle layer between an AI assistant and internal software-engineering knowledge. The assistant connects to UCE MCP, not directly to the raw graph backend. UCE then performs impact analysis, risk assessment, explanation, file mutation, and RBAC authorization using graph-backed context. This boundary is important because direct database access would allow an assistant to bypass the policies that UCE is supposed to enforce.

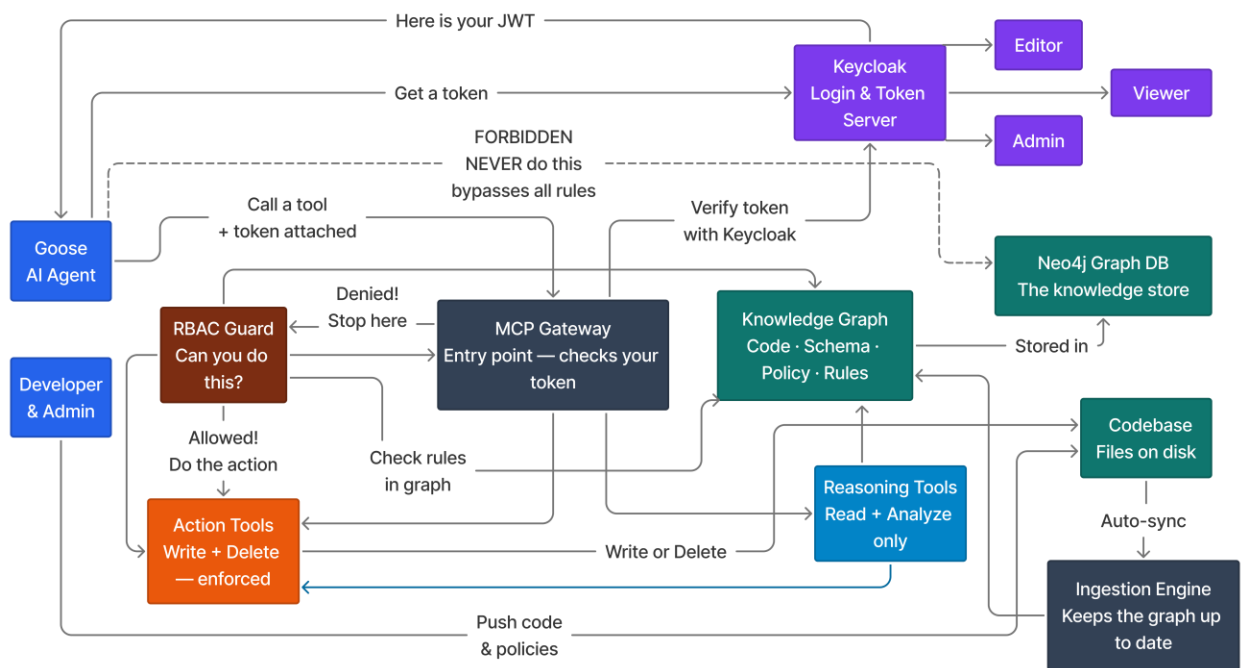


Figure 1. High-level UCE architecture showing assistant, RBAC guard, MCP gateway, reasoning/action tools, Keycloak identity, Neo4j graph storage, and protected data paths.

The implementation uses a graph schema with nodes for File, Table, Column, Requirement, Policy, Function, Class, Method, Identifier, Role, and AuthorityRule. Important relationships include File imports, File uses table, File references column, Requirement governs table/column, Policy enforces requirement, File declares function/class, Class has method, Function calls function, Policy defines authority rule, and AuthorityRule requires role. This schema is intentionally explicit: it makes the reasoning path visible instead of relying on model memory.

Table 1. Core UCE graph schema concepts.

Graph element	Purpose
File, Function, Class, Method	Represent code structure and enable impact analysis over implementation artifacts.
Table, Column	Represent database schema objects parsed from the target project.
Requirement	Represent software requirements such as required schema fields and audit behavior.
Policy	Represent governance rules that enforce one or more requirements.
Role, AuthorityRule	Represent RBAC authorization over path patterns and operations.
IMPORTS, USES_TABLE, REFERENCES_COLUMN	Connect source code to code dependencies and schema entities.
GOVERNS, ENFORCES	Connect governance artifacts to technical implementation entities.

Ingestion was split into deterministic and LLM-assisted lanes. Deterministic ingestion handles schema parsing and code parsing without requiring an LLM. Requirement and policy documents can be represented in Markdown and mapped into graph entities. The LLM-assisted lane is optional and useful for extracting richer links from less structured documents, but the final evaluation emphasizes deterministic governance links so that results are reproducible.

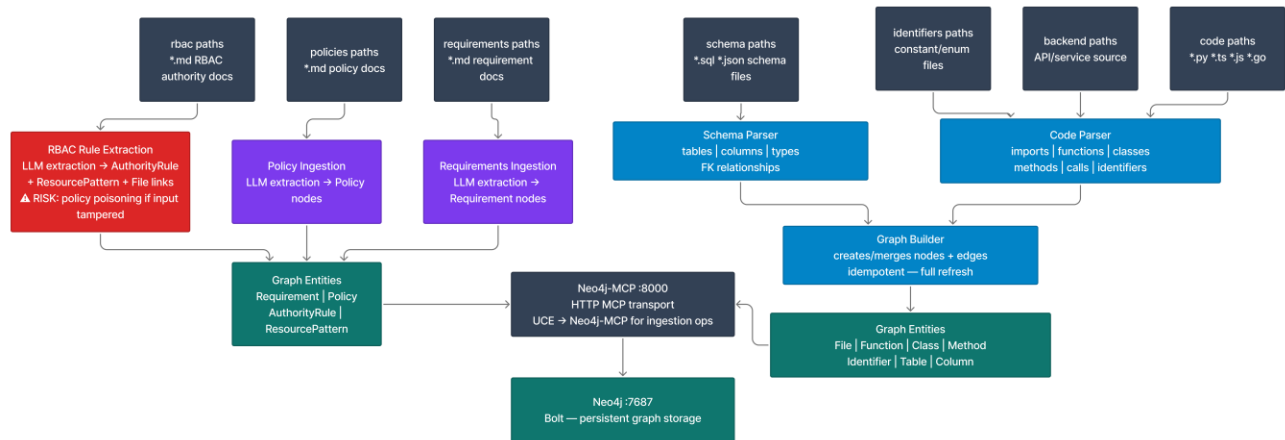


Figure 2. Ingestion architecture comparing deterministic parsing for schema/code with optional LLM-assisted extraction for requirements, policies, and RBAC documents.

The MCP layer exposes reasoning and mutation tools. Reasoning tools include `impact_analysis`, `explain_change`, `risk_assessment`, `count_functions_in_file`, and `find_identifier_usage`. Mutation tools include `authorize_change`, `write_file`, and `delete_file`. The mutation path checks identity and RBAC rules before writing, while the reasoning path can return affected files, implicated requirements, enforced policies, function counts, and risk scores.

B. External components that you used

Neo4j: Graph database used to store code, schema, requirement, policy, and RBAC relationships.

Model Context Protocol pattern: Used as the tool boundary between the assistant and UCE.

Keycloak: Used to model identity, role tokens, and viewer/editor/admin authentication flows.

llama3:instruct: Local small LLM used for the corrected no-tool baseline because the project prioritizes data privacy and hardware-feasible local inference.

Python: Primary implementation language for ingestion, graph interaction, RBAC evaluation, MCP server logic, and benchmarks.

TypeScript/Next.js target project: Representative project used for schema/code/governance evaluation.

pandas and matplotlib: Used for benchmark tables, summaries, and result visualizations.

C. Design of your empirical studies

The empirical study compares two systems. The first is a real no-tool local LLM baseline using `llama3:instruct`. This model receives pasted static context and returns JSON predictions, but it has no access to MCP, Neo4j, `grep`, filesystem tools, or graph queries. The second is MCP-UCE, which uses the graph-backed context engine and RBAC guard. The benchmark asks whether each system can identify affected files, implicated requirements, enforced policies, and RBAC allow/deny decisions.

The scenario benchmark contains 24 software-impact scenarios: table-level changes, column-level changes, and file-level refactors. For each scenario, the output is scored using exact overlap against oracle file paths, requirement IDs, and policy IDs. The RBAC benchmark contains 40 probes across viewer/editor roles, write/delete operations, governance paths, backend paths, and other paths. A breach occurs when the oracle says a request should be denied but the system predicts that it is allowed.

Metrics include precision, recall, and F1 for affected files, requirements, and policies; caught-any rate for requirement and policy violations; missed-all rate for scenarios with non-empty truth labels; and RBAC breach rate on oracle-denied probes. This design intentionally evaluates governance behavior rather than only generic answer quality.

5. Results

A. Empirical results

The corrected experiment used `llama3:instruct` as a real local no-tool LLM baseline. The baseline produced raw JSON responses for each benchmark batch. The evaluator then parsed requirement IDs, policy IDs, file paths, and RBAC decisions from those actual model responses. This is different from a lexical or keyword baseline: the prediction came from the model response, and parsing was used only to extract the structured IDs needed for scoring.

Table 2. Corrected scenario-level quality comparison using llama3:instruct for the no-tool LLM baseline.

System	Scenarios	Req. F1	Req. caught-any	Policy F1	Policy caught-any	File F1
No-tool local LLM	24	0.333	55.0%	0.340	36.8%	0.346
MCP-UCE	24	0.486	77.3%	0.603	71.4%	0.791

Latency is intentionally not included as a direct comparison in this table because the two measured scopes are different. The no-tool local LLM latency measures `llama3:instruct` prompt processing and JSON generation. The MCP-UCE latency in the benchmark artifact measures deterministic backend graph/tool execution after a tool has been invoked; it does not include a local LLM deciding to call the tool or generating the final natural-language answer. Therefore, these numbers should not be interpreted as evidence that tool-calling Llama is faster than no-tool Llama.

Table 3. End-to-end 24-scenario latency comparison.

Mode	Scenarios	Total time	Mean per scenario	Notes
No-tool llama3:instruct	24	843.0 s	35.1 s	Single local LLM generation path using pasted static context.
Routed tool-assisted llama3:instruct + UCE output	24	1269.0 s	52.9 s	Orchestrator calls UCE-style <code>impact_analysis</code> , then local LLM emits final structured output.

The fair end-to-end latency comparison shows that routed tool-assisted output was slower in this local run: 1269.0 seconds versus 843.0 seconds for the no-tool baseline, a difference of 426.0 seconds over 24 scenarios. This does not weaken the quality/safety result; it means UCE improves policy, requirement, and RBAC behavior at the cost of additional local generation time when the final answer copies large tool-derived result arrays.

The no-tool LLM caught at least one implicated requirement in 55.0% of scenarios that had requirement truth labels, while MCP-UCE caught at least one in 77.3%. The no-tool LLM caught at least one implicated policy in 36.8% of relevant scenarios, while MCP-UCE did so in 71.4%. MCP-UCE also had higher F1 for files, requirements, and policies. This supports the main hypothesis: a local LLM with static pasted context is useful, but deterministic graph context improves governance-aware coverage.

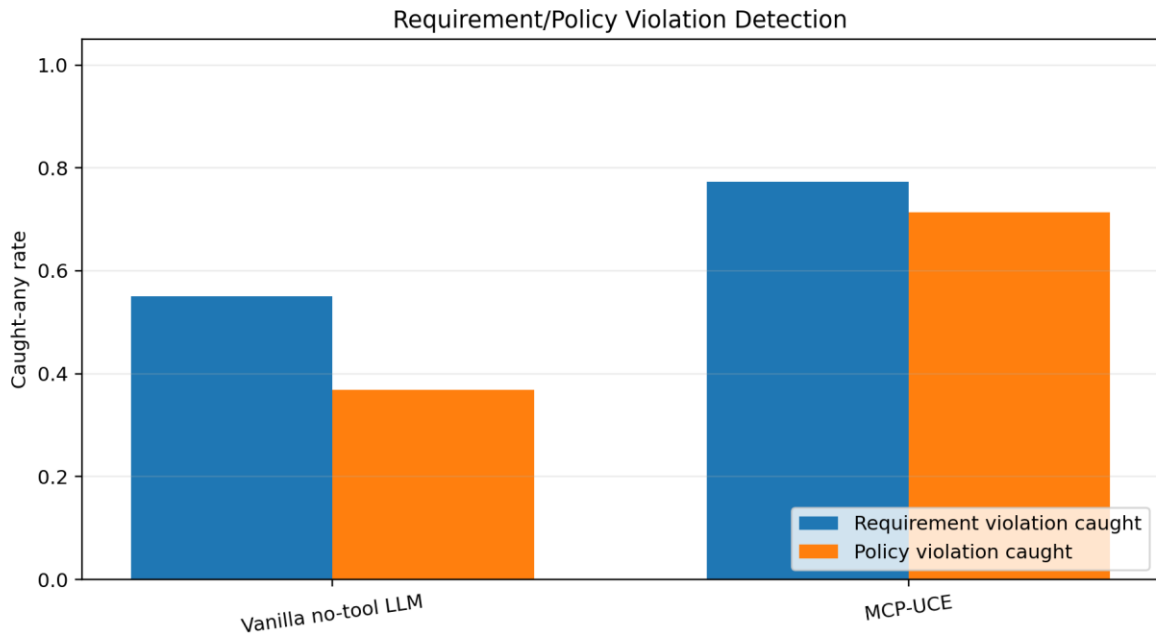


Figure 3. Requirement and policy violation caught-any rates for the real no-tool local LLM baseline versus MCP-UCE.

Table 4. RBAC probe results.

System	Probes	Denied truth	Breaches	Breach rate	Blocked denied	Blocked rate	False deny
No-tool local LLM	40	34	22	64.7%	12	35.3%	0
MCP-UCE	40	34	0	0.0%	34	100.0%	0

The RBAC result is the clearest safety result. On 34 oracle-denied probes, the no-tool local LLM allowed 22 requests that should have been denied, producing a 64.7% breach rate. MCP-UCE blocked all 34 denied operations, producing a 0.0% breach rate and no false denials. This demonstrates why RBAC should be enforced by the tool gateway rather than left to prompt compliance.

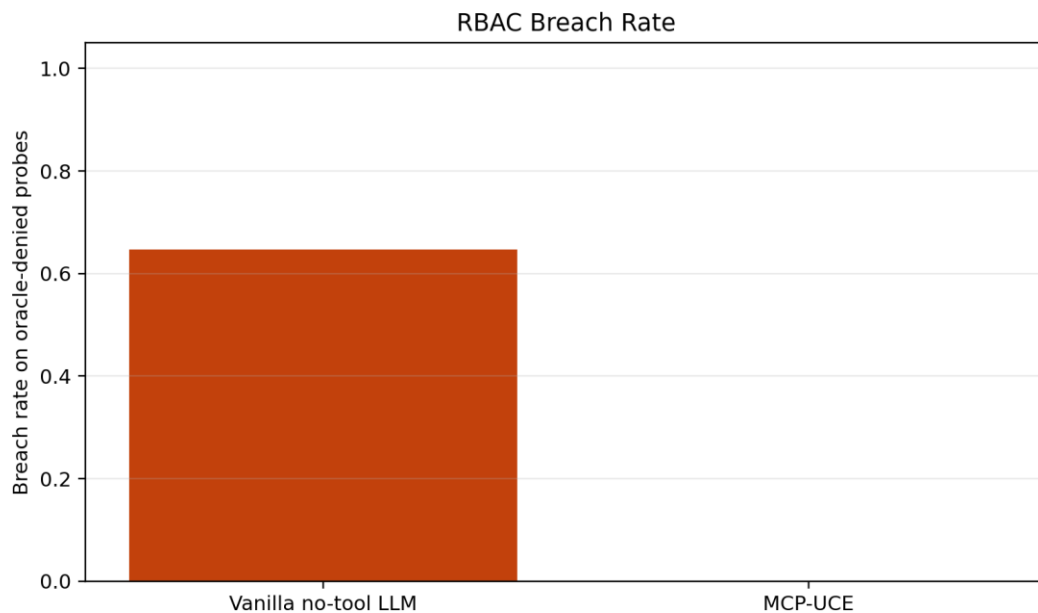


Figure 4. RBAC breach rate on oracle-denied probes for the real no-tool local LLM baseline versus MCP-UCE.

Table 5. Per-scenario no-tool local LLM governance detection outcomes.

Scenario	Type	Req. caught/true	Policy caught/true	Req. F1	Policy F1
TBL-account	table	1/4	1/2	0.400	0.667
TBL-agents	table	1/4	1/2	0.400	0.667
TBL-meetings	table	3/6	2/3	0.667	0.800
TBL-session	table	1/4	0/2	0.400	0.000
TBL-user	table	1/5	0/3	0.333	0.000
TBL-verification	table	1/3	0/2	0.500	0.000
COL-account-access_token	column	1/1	0/0	1.000	0.000
COL-account-access_token_expires_at	column	0/0	0/0	0.000	0.000
COL-agents-created_at	column	0/1	0/1	0.000	0.000
COL-agents-id	column	0/0	0/0	0.000	0.000
COL-meetings-agent_id	column	1/1	1/1	1.000	0.667
COL-meetings-created_at	column	1/1	1/1	0.667	0.667
COL-session-created_at	column	0/1	0/1	0.000	0.000
COL-session-expires_at	column	0/2	0/1	0.000	0.000
COL-user-created_at	column	0/1	0/1	0.000	0.000
COL-user-email	column	1/1	1/1	1.000	1.000
COL-verification-created_at	column	0/1	0/1	0.000	0.000
COL-verification-expires_at	column	1/2	1/1	0.667	1.000
FIL-src_trpc_init.ts	file	0/8	0/4	0.000	0.000
FIL-src_trpc_query-client.ts	file	0/1	0/1	0.000	0.000
FIL-src_trpc_routers_app.ts	file	0/6	0/3	0.000	0.000
FIL-src_inngest_client.ts	file	0/8	0/4	0.000	0.000
FIL-src_app_api_auth [...all]_route.ts	file	0/0	0/0	0.000	0.000
FIL-src_app_api_inngest_route.ts	file	0/0	0/0	0.000	0.000

The per-scenario breakdown shows a pattern: the no-tool LLM often identified obvious table or column requirements when the schema names appeared directly in the requirement text, such as `account.access\_token` and `user.email`. It struggled more on transitive file-level scenarios and on policies that required following requirement-to-policy relationships. This is exactly the case where a graph helps: file impact, schema references, imports, requirements, and policies are multi-hop relationships, not isolated keywords.

B. Software deliverables, if any

The software deliverables are substantial for a course prototype. UCE includes a runnable ingestion and reasoning system, a graph schema, MCP server tooling, RBAC enforcement, Keycloak bootstrap scripts, Docker-oriented setup documentation, evaluation scripts, and generated reports/figures. The project also produced a corrected real LLM baseline runner so future experiments can avoid confusing lexical proxy behavior with actual model behavior.

- Runnable UCE CLI and MCP server implementation.
- Graph ingestion for source files, imports, functions, classes, database schema, requirements, policies, and RBAC rules.
- Graph-backed impact analysis and risk assessment tools.
- RBAC-gated write/delete operations with viewer/editor/admin role semantics.
- Benchmark artifacts for 24 impact scenarios and 40 RBAC probes.
- Word-report-ready architecture diagrams, empirical figures, and result tables.

6. Waiting room

A. Things you wanted to do but could not because of time limitation

There are several extensions I wanted to complete but could not fully finish within the course timeline. The first is a larger evaluation across more repositories and more governance documents. The current benchmark is useful because it checks concrete requirement IDs, policy IDs, and RBAC probes, but a stronger study would include multiple projects, more schemas, more ambiguous requirements, and repeated runs with several local LLMs.

- Evaluate larger local models and compare them under identical privacy constraints.
- Add repeated trials to measure output variance from the no-tool LLM baseline.
- Expand the requirement and policy corpus beyond the initial Markdown governance documents.
- Add an IDE or chat UI so developers can inspect graph evidence interactively.
- Implement richer LLM-assisted ingestion with evidence spans and confidence scores while keeping deterministic verification for high-risk edges.
- Add stronger production hardening: audit logs, policy versioning, signed governance documents, and secure deployment profiles.
- Run a user study comparing how quickly developers identify impacted requirements using UCE versus manual search.
- Add more advanced GraphRAG retrieval strategies, including community summaries and mixed global/local graph search.

The most important future direction is to make UCE not only a reasoning engine but a daily developer workflow. Ideally, a developer would ask an assistant to make or explain a change, and the assistant would automatically show impacted files, requirements, policies, RBAC authorization, and graph evidence before proposing code. That would turn policy compliance from a late manual review step into an always-on part of development.

References

8. Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. NeurIPS 2020. <https://papers.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
9. Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. <https://arxiv.org/abs/2107.03374>
10. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
11. Microsoft Research. Project GraphRAG. <https://www.microsoft.com/en-us/research/project/graphrag/>
12. Anthropic. (2024). Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>
13. Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>
14. Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761. <https://arxiv.org/abs/2302.04761>
15. NIST. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). <https://doi.org/10.6028/NIST.AI.100-1>
16. Meta. (2024). Meta-Llama-3-8B-Instruct model card. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>

17. Ollama. Llama 3 model library entry. <https://ollama.com/library/llama3>